Heart Disease Prediction — MLOps Pipeline

End-to-End MLOps Group Project Report

github.com/rahmaaroua/heart-disease-mlops

Ali Belhrak & Rahma Aroua

April 1, 2026

1. Problem Description

Cardiovascular disease remains one of the leading causes of mortality worldwide. Early and accurate detection enables timely clinical intervention and improved patient outcomes. This project builds a production-ready MLOps pipeline that trains, tracks, containerises, and deploys three machine learning classifiers on the UCI Heart Disease dataset, serving predictions through a web application backed by a documented REST API.

1.1 Dataset

The UCI Heart Disease (Cleveland) dataset originates from the landmark study by Detrano et al., *"International application of a new probability algorithm for the diagnosis of coronary artery disease"* (cited 710+ times), available at `https://archive.ics.uci.edu/ml/datasets/heart+Disease`. It contains **303 patient records** with **13 clinical features** (age, sex, chest pain type, resting BP, cholesterol, fasting blood sugar, resting ECG, max heart rate, exercise-induced angina, ST depression, slope, vessels coloured, thalassemia) and a binary target (disease present / absent). Its real clinical measurements and well-defined outcomes make it an ideal benchmark for cardiovascular classification tasks.

Aspect	Details
Task	Binary classification: disease present (1) vs. absent (0)
Class imbalance	Moderate; handled via `scale_pos_weight` in XGBoost and `StratifiedKFold` cross-validation
Preprocessing	Categorical label encoding, `StandardScaler` fitted on training split only (prevents data leakage), 80/20 train–test split

1.2 Champion Metric: F1-Score

Accuracy is misleading on imbalanced medical data. A false negative (missed disease) carries a higher clinical cost than a false positive. F1-score balances precision and recall and is therefore used as the champion selection criterion. The CI/CD pipeline enforces a minimum F1 threshold of **0.70** before any model is promoted to production.

2. System Architecture

The system is structured as a seven-layer pipeline where each layer corresponds to a dedicated MLOps component. Data flows from raw CSV through preprocessing and model training into an MLflow model registry, then into a Dockerised inference service orchestrated by Kubernetes, with the entire workflow automated by GitHub Actions.

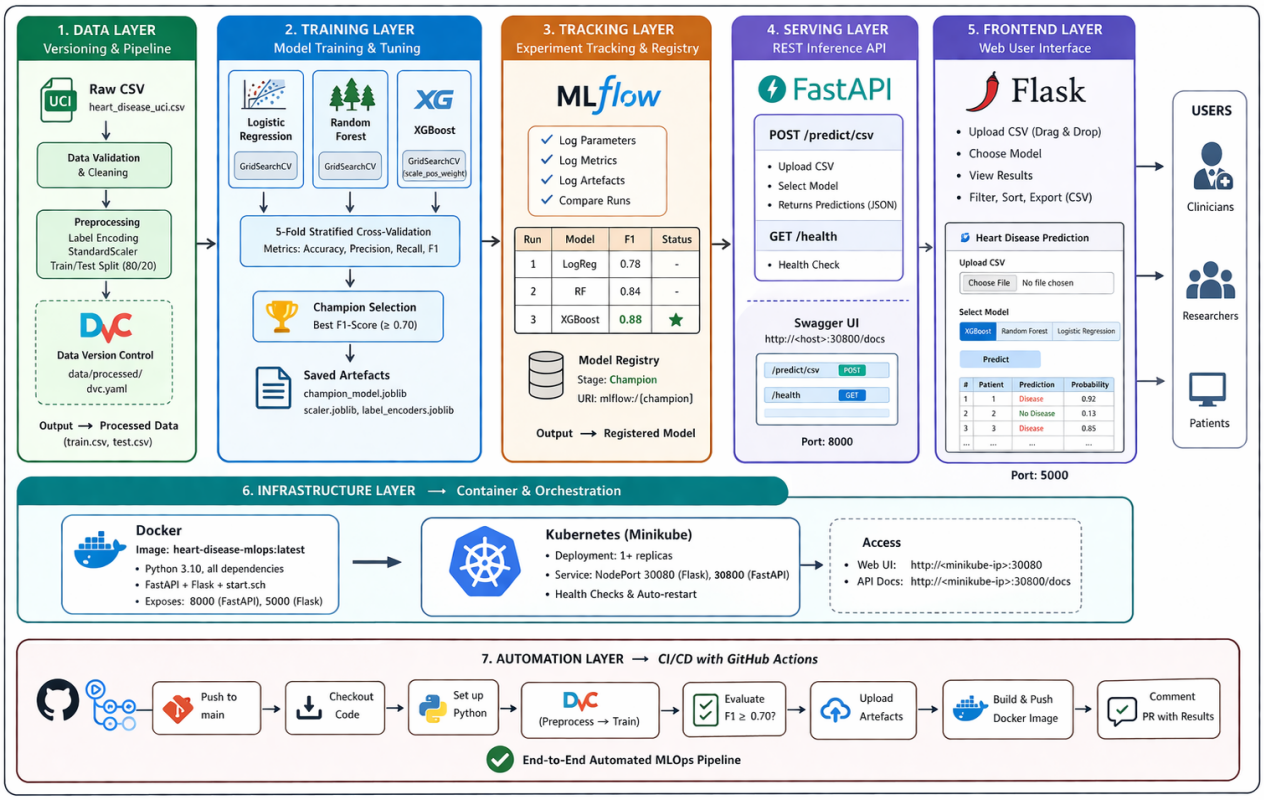

Figure 1: System Architecture Diagram — end-to-end pipeline overview (Clean Version)

2.1 Pipeline Layers

Layer	Component	Technology	Artefacts Produced
Data	Versioning & DAG	DVC	`data/processed/`, `dvc.yaml`
Training	Model training & tuning	scikit-learn, XGBoost, GridSearchCV	`champion_model.joblib`
Tracking	Experiment logging	MLflow	`mlruns/`, model registry
Serving	REST inference API	FastAPI (port 8000)	`POST /predict/csv`
Frontend	Web UI	Flask (port 5000)	Upload → results table
Infra	Containerisation + orch.	Docker + Kubernetes	Deployment & Service YAML
Automation	CI/CD	GitHub Actions	Train → Evaluate → Build → Push

Two services share the same Docker container: **FastAPI** handles stateless ML inference (auto-documented via Swagger at `/docs`), while **Flask** serves the HTML web UI. NodePort `30080` exposes Flask and `30800` exposes FastAPI inside Kubernetes.

3. Tools Used and Justification

3.1 DVC

Git cannot efficiently track large binary files such as CSVs and serialised models. DVC stores these in a remote while committing only lightweight `.dvc` pointer files to Git. Its pipeline DAG (`dvc.yaml`) ensures each stage re-runs only when upstream inputs change — a single `dvc repro` fully reconstructs the experiment from scratch.

3.2 MLflow

MLflow logs every run's parameters, metrics, and artefacts in a unified UI, enabling side-by-side comparison of all three models. The model registry decouples training from deployment: the inference API loads the champion by registered URI, so promoting a new champion requires no code change.

3.3 scikit-learn + XGBoost

Three classifiers were trained: **Logistic Regression** (interpretable baseline), **Random Forest** (robust low-variance ensemble), and **XGBoost** (gradient-boosted trees with `scale_pos_weight` for class imbalance). All were tuned with `GridSearchCV` and `StratifiedKFold` (5 folds) to preserve class ratios per fold. XGBoost achieved the highest F1 and was selected as champion.

3.4 FastAPI + Flask

FastAPI was chosen for the inference backend for its automatic OpenAPI/Swagger generation, async support, and native multipart upload (`POST /predict/csv`). Flask handles the HTML frontend; results are stored server-side in a `RESULT_STORE` dictionary to avoid the 4 KB browser cookie limit.

3.5 Docker

Containerisation eliminates environment inconsistencies. The Dockerfile packages both services and all dependencies; `start.sh` launches FastAPI first, polls its health endpoint, then starts Flask — guaranteeing correct startup order.

3.6 Kubernetes (Minikube)

A `Deployment` manifest controls replicas and rolling updates; a `Service` manifest of type `NodePort` exposes both ports. This mirrors real production orchestration (health probes, declarative desired state) on a local cluster.

3.7 GitHub Actions

On every push to `main`, the workflow automatically: preprocesses data, trains all three models, enforces F1 ≥ 0.70, uploads artefacts, builds and pushes the Docker image to Docker Hub, and posts a model comparison table as a PR comment — zero manual steps required.

4. Challenges Faced

Missing `python-multipart` dependency
FastAPI requires this package at import time for file upload routes; its absence caused a hard crash before any request was served. *Fix:* added explicitly to `requirements.txt`.

Relative path resolution
`api/main.py` used relative paths that broke when launched from a different working directory. *Fix:* all paths anchored with `Path(__file__).parent.parent`.

Flask template discovery from a subfolder
Running Flask from `api/` raised `TemplateNotFound` errors. *Fix:* `start.sh` and the Dockerfile both launch Flask from the project root with the correct `PYTHONPATH`.

Kubernetes image pull in Minikube
Minikube's internal daemon is isolated from the host, making locally built images invisible inside the cluster. *Fix:* image pushed to Docker Hub; `deployment.yaml` references the remote tag.

Scaler data leakage
Fitting `StandardScaler` before the train–test split would leak test-set statistics, inflating evaluation scores. *Fix:* scaler fitted on the training split only; test split transformed via `transform()` alone.

Repository: alibelhrak/heart-disease-mlops **Stack:** DVC • MLflow • scikit-learn • XGBoost • FastAPI • Flask • Docker • Kubernetes • GitHub Actions